

# Ahmad: Away from Home

---

## Ahmad: Away from Home (*student proposal*)

---

**Assessment:** ✓ Fit · **Category:** Games · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

**Proposal (as submitted):** *Ahmad: Away from Home* is a **puzzle-based exploration game** (in the style of a map-piecing game): players **zoom out to a world map** to place and connect puzzle pieces, then **zoom in to explore each piece as a fully navigable environment**, hunting for clues and more pieces until a completed section reveals the way out. Named load-bearing C++: a game loop handling **player input, NPC AI, collision detection, puzzle fitting and game-state transitions** in real time, maintaining a consistent map layout from the pieces with instant feedback. Named technical features: an **Undo/Redo puzzle-placement system** (track placements, undo/redo without losing collected pieces/progress); **Real-Time World & Progression Synchronisation** (discoveries during exploration — clues, pieces, objectives — update the world map live); **Automatic Terrain-Connection Detection** (auto-identify connected same-terrain areas, plus **tile preview + placement prediction** showing valid positions and resulting connections); and a **Multi-State Gameplay System** (smooth transition from the 2D puzzle-piecing view to the top-down exploration view). Tool: a **Game-State Debugging Tool** to visualise the current puzzle map, found pieces, objectives and exploration states, keeping pieces + environments in sync and cutting manual setup. Design features: a **rotatable piece-fitting mechanic** (rotate + place pieces into matching shapes, any order as long as the shape fits); **seamless zoom transition**; an **environmental clue system** (≥4 NPCs + environmental signs — worn paths, notes, landmarks — giving subtle non-handholding hints; ≥4 distinct puzzle-piece **biomes** — water, forest, ruins... — each with unique interactables/hazards); and **free-roam exploration within each piece**. Team: **6 CS + 1 Design + 1 Artist**.

*Note (module completeness):* the proposal doesn't name a **platform** — the zoom/place/rotate interaction is touch-friendly, so it's planned as a mobile build (Native Android, Pixel 9a; a Web-on-mobile build is a viable alternative). It also doesn't

mention an **External Database** or accounts, but the module mandates Firebase — plan to persist save games / progress there (a standard, small addition; see the External Database row + M1G12-style integration). Neither is a category concern.

*References:*

- [Carto on Steam](#) (a tile-placement adventure where you discover pieces of the world map, **rearrange and rotate them (edges must match) to shape the levels**, then explore them on foot — an almost feature-for-feature precedent for the place-and-connect-then-explore loop)
- [Dorfromantik — Wikipedia](#) (a tile-placement puzzle of placing/rotating tiles so **terrain edges match**, forming connected landscape groups — the automatic-terrain-connection + placement-prediction precedent)
- [Gorogoa on Steam](#) (a puzzle of **arranging and zooming into panels/tiles** to connect them and open paths — the seamless-zoom + piece-arrangement precedent)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

| Core Tech Requirement                                       | Ahmad: Away from Home |
|-------------------------------------------------------------|-----------------------|
| External Database                                           | ✓                     |
| Authentication »                                            | ●                     |
| Analytics »                                                 | ●                     |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓                     |
| Application Communication                                   | ✓                     |
| Continuous Integration                                      | ✓                     |
| Data-Driven Architecture                                    | ✓                     |
| Front-end + Back-end                                        | ✓                     |
| Custom Tools Development                                    | ✓                     |
| 2D Graphical Output                                         | ✓                     |
| 3D (strong students only)                                   | ✕                     |
| Networking System                                           | ●                     |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — zoom/place/rotate + top-down movement are touch-first; a Web-on-mobile (Emscripten→WASM) build is a viable alternative. The Native PC build is the dev/tools vehicle (the game-state debug + environment editor). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for save-game / progress persistence.

**Front end:** A 2D game with two views — the zoomed-out world map (puzzle pieces + connection previews) and the zoomed-in top-down explorable environments (Ahmad, NPCs, interactables, clues) — with a seamless zoom transition and the objective/clue HUD. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; some screens (menus) can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

**Back end:** The real-time C++ engine — player input, the puzzle-fitting + rotation + placement-validation, automatic terrain-connection detection, the undo/redo command stack, NPC AI + top-down collision, the multi-state (map↔exploration) machine and the world/progression sync — with instant feedback; plus Firebase for save/progress. **Languages:** C++ (engine); Firebase-backed services on the Extended tier.

**Backend scope:** Local now, server-ready (T2) — the save/progress seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — it validates + rotates + fits puzzle pieces, runs automatic terrain-connection detection, maintains the undo/redo command stack, drives NPC AI + collision in exploration, and transitions the multi-state (map↔walk) machine with instant feedback.
- *Data-driven:* the puzzle pieces + their terrain edges, the  $\geq 4$  biomes and their interactables/hazards, the  $\geq 4$  NPCs + clue placements, and objectives are all data — a new piece, biome or clue is a data change (authored in the debug/environment tool).
- *2D:* both views (the world-map puzzle + the top-down exploration) and the zoom transition are 2D and custom-rendered. *3D:* not needed.
- *Custom tool:* the **Game-State Debugging + environment tool** — visualise the puzzle map, found pieces, objectives and exploration states, and author pieces/environments/clues, keeping map + environments in sync.
- *Networking:* single-player; save/progress is async client↔cloud (Application Communication ✓), so a Networking System is optional (●). *External DB:* save games / progress (Firebase — add per the module). *Auth » / Analytics* »: module-deferred to T2 (●).

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the loop + puzzle-fitting + the **undo/redo** command stack.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a C++ loop handling input, puzzle fitting/rotation/placement-validation, an undo/redo command stack (progress-preserving), and game-state transitions with instant feedback; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the tile/terrain system (M1G14) and camera/state (M1G16) build on it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — incl. the world/progression sync model.

Tech & dependencies

- **Tech:** An entity/data model with puzzle pieces + terrain edges, the  $\geq 4$  biomes + interactables/hazards, the  $\geq 4$  NPCs + clues, and objectives in hot-reloadable JSON; a shared world/progression state that exploration discoveries update live and the map reads back.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the debug/environment tool), M1G14 (tile/terrain), M2G13 (progression/clue systems).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (piece connections, terrain groups, objective/exploration state), Tracy profiling of a large connected map + a busy exploration scene, Catch2 tests (placement validation, terrain-

connection detection, undo/redo, data loading), on a GitHub Actions matrix — the base of the Game-State Debug Tool.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G2 (the full debug/environment tool); M2G3 (CI hardening).

- **G4 2D Graphics** — the world-map view, the top-down exploration view, and the zoom.

Tech & dependencies

- **Tech: Web build:** A sprite/tile renderer for the zoomed-out puzzle map (pieces + connection previews) and the zoomed-in top-down environments (Ahmad, NPCs, interactables), a camera, and the objective/clue HUD; SDL2/3/OpenGL ES, GLSL, stb\_image, FreeType.
- **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (biome visual variety + zoom polish); the debug tool shares this renderer.

- **G14 Level / Tile Map System** — the **puzzle tiles + automatic terrain-connection detection + placement prediction**.

Tech & dependencies

- **Tech:** A tile/piece map (JSON) with terrain edges + collision data, placement validation (shape/edge fit, any rotation), a connected-component/flood-fill pass that auto-detects same-terrain groups, and a tile-preview that shows valid positions + resulting connections before commit.
- **Prerequisites:** M1G2 (piece/terrain data model), G4 (rendering).
- **Feeds into:** The core puzzle mechanic; the explorable environments as tile maps.

- **G16 Camera, Scene & State Management** — the **seamless zoom + the multi-state** transition.

Tech & dependencies

- **Tech:** Interpolation/easing for the zoom in/out, a scene/state stack managing the 2D puzzle-piecing view ↔ the top-down exploration view, and clean transitions that preserve world/progression state.
- **Prerequisites:** G4 (rendering), G1 (state machine).

- **Feeds into:** The signature multi-state gameplay; polished navigation.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — on-device perf + the save/progress seam.

Tech & dependencies

- **Tech: Web build:** Memory pools, Tracy profiling of a large connected map + busy exploration, ASan on MSVC, and on-device validation (Pixel 9a) holding the frame budget through zoom transitions; the Firebase/Linux-server seam (save/progress) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + smooth zoom/exploration.

- **G2 ★ Content Editor — Core (Game-State Debug + Environment Tool)** — visualise/author the map, pieces, objectives, exploration states.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: visualise the puzzle map + found pieces + objectives + exploration state, and author pieces (terrain edges), biomes, clues and NPC placements, with round-trip JSON, live preview and undo/redo — keeping map + environments in sync.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
- **Prerequisites:** G2 (data model), M1G3 (debug overlays), M1G14 (tile/terrain).
- **Feeds into:** All content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (placement validation, terrain-connection, undo/redo, save/load) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where

realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

• **G4 2D Graphics — Advanced** — the  $\geq 4$  biome visual identities + zoom/lighting polish.

Tech & dependencies

- **Tech: Web build:** OpenGL/GLSL shaders, lighting/atmosphere per biome (water/forest/ruins...), post-FX for the zoom transition and texture atlases. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** M1G4 (2D graphics), M2G1 (optimization).
- **Feeds into:** The biome variety + polished zoom at showcase.

• **G6 Content Editor — Advanced Tool Features (Environment & Clue Authoring)** — the biome/clue/NPC authoring matured.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets: drag-drop environment building per biome, place interactables/hazards, clues (worn paths/notes/landmarks) and NPCs with hint links, and live-preview the map↔environment sync; deep undo/redo.
- **Prerequisites:** M2G2 (editor core).
- **Feeds into:** Fast environment/clue authoring; the content showcase.

• **G13 Gameplay Systems II** — objectives/progression, the **clue system**, biome interactables/hazards + NPC hints.

Tech & dependencies

- **Tech:** C++ systems over the entity/data model: the objective/progression tracking + world-map reveal, the environmental clue system (subtle non-handholding hints), the  $\geq 4$  biomes' unique interactables/hazards, and the  $\geq 4$  NPCs' hint behaviours; balancing-data-driven.
- **Prerequisites:** M1G2 (data), M1G14 (tiles/terrain).
- **Feeds into:** The exploration depth at showcase; T2 content expansion.

## Track Goals & Team Composition

**Recommended composition:** 6 CS + 1 Design + 1 Art (8) — **as proposed**, and well-matched: an exploration game leans on its **environments** ( $\geq 4$  distinct biomes, Ahmad +  $\geq 4$  NPCs, interactables, the world-map art, UI), which one dedicated artist can carry, while a single designer owns the FDD, the puzzle/mechanic design, the biome/clue/level design and the exploration pacing (rules: min 1 Design mandated, max 9 total,  $D+A \leq 3$ ; see `team-formation.pdf`). If the biome + clue content outgrows one designer, consider a 2nd designer. *Scope note (advisory):* a puzzle-fitting + terrain-connection engine + a multi-state/zoom system + free-roam exploration with NPCs + a debug/environment tool is a solid, cohesive scope — the place/rotate/connect mechanic, the seamless zoom, and **one or two fully explorable biomes** with basic clues is the M1/M2 floor; all  $\geq 4$  biomes with unique interactables/hazards, the full  $\geq 4$ -NPC clue web and the polished world reveal are the stretch. The **automatic terrain-connection detection + placement prediction** is the trickiest CS bit — validate it early on a small piece set.

**Track goals** — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the place-connect-then-explore USP, the rotatable piece-fitting, the seamless zoom, the clue system and the  $\geq 4$  biomes; primary benchmark e.g. Carto / Dorfromantik, plus a Singapore benchmark)
- M1D8 Puzzle & Mechanic Design (the rotatable piece-fitting + terrain-connection mechanic — a catalogue of piece/connection situations, a difficulty ramp, and 5–8 designed puzzle layouts authored to the CS data format)
- M1D5 Level & Progression Design (the  $\geq 4$  biomes, the exploration + clue placement, the non-handholding hint design and the world-reveal pacing)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D2 Working Prototype — Full-Content Build (the polished place → zoom → explore experience across the shipping biomes, in the team's engine + editor)

- **UI/UX M1** (graded by the Design instructor for the Games category):
  - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the piece-placement/rotation UI, the connection-preview, the zoom affordance and the objective/clue HUD, plus a wireframe baseline.
- **UI/UX M2:**
  - M2U1 ★ Functional UI & Usability — the map/placement view, exploration HUD, clue log and objectives navigable unaided on the Native-Android target.
  - M2U2 ★ UX Validation & Final Polished UI — validated, polished place-rotate-explore + zoom UX tested with players on the Pixel 9a (is the piece-fitting + zoom legible and satisfying?).
- **Art M1:** 1 artist (Ahmad +  $\geq 4$  NPCs +  $\geq 4$  biomes + interactables + UI)
  - M1A1 ★ Art Bible / Proof of Concept (the "away from home" visual direction — tone, palette, the biome + world-map identity language)
  - M1A2 Character Concepts & Model Sheets (Ahmad + the  $\geq 4$  NPCs)
  - M1A3 Environment, Props & VFX Concepts (the  $\geq 4$  biomes — water/forest/ruins... — their interactables/hazards, and the world-map puzzle-piece art)
- **Art M2:**
  - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
  - M2A2 Character Animation — Extensive (Ahmad's movement + NPC idles/gestures)
  - M2A3 Environment Alive — Ambient Animation & Layering (living biomes — water, foliage, ruins ambience — that make each piece feel explorable)
- **TEAM M1:**
  - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (world-map + per-biome ambience, piece-place/rotate/connect cues, clue-found, zoom-transition), suitable and correctly triggered.
  - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

• **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (world +  $\geq 4$  biome ambiences + place/connect / clue / discovery / zoom / NPC SFX,  $\geq 6$  types), scaling with exploration.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct per-biome ambience + connect/discovery cues).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the Game-State Debug + environment tool), prototype demo, team post-mortem (wk14)

---

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-22.*